# Q4: Web Services, APIs, REST and JSON, Location and Gestures

** = supplementary, not in the lecture materials · Web service vs API · SOAP vs REST · HTTP methods · JSON vs XML · parsing · GET and POST · Postman · mock APIs · location permissions · sensors and gestures

## 1. Web services

A **web service** is software that lets **two applications talk to each other over a network** (usually the internet) using **standard, agreed-upon rules**.

**Your Android app (client) → Request → Web service on a server → Response → back to the app**

| Feature | Meaning |
|---|---|
| **Platform-independent** | Works across different operating systems and programming languages |
| **Network-accessible** | Client and server can be anywhere |
| **Standard data formats** | Data is exchanged as **JSON** or **XML** so both sides understand it |
| **Self-contained requests** | Each request carries everything the server needs to respond |

## 2. APIs

An **API (Application Programming Interface)** is a **contract** that lets one piece of software **request something from another without needing to know how it works inside**.

**Restaurant analogy (lecture):** You (the **client**) place an order without knowing how it is cooked → the **waiter (the API)** carries the request to the kitchen and brings the dish back → the **kitchen (the server)** does the actual work.

Example: App asks “Get me the weather” → API passes the request → Server returns the weather data.

### Web service vs API

**Every web service is a kind of API, but not every API is a web service.** API is the broader idea.

| Criteria | API | Web Service |
|---|---|---|
| Scope | Any interface that lets software request functionality, over a network or not | A specific kind of API that always works over a network |
| Network required? | Not always (for example a local Android SDK call) | Always |
| Protocol | No fixed rules | Follows a defined protocol such as SOAP or REST |
| Example | Android’s **Camera API** (no internet needed) | A REST weather endpoint at a URL |

## 3. How an HTTP request and response work

Apps talk to web APIs using **HTTP** (HyperText Transfer Protocol), a request/response protocol.

**Parts of a request** **:

| Part | Meaning | Example |
|---|---|---|
| **Endpoint (URL)** | Address of the resource | `https://api.example.com/users` |
| **Method** | The action | `GET`, `POST`, `PUT`, `PATCH`, `DELETE` |
| **Headers** | Extra information (metadata) | `Content-Type: application/json`, `Authorization: Bearer <token>` |
| **Query parameters** | Options added to the URL after `?`, joined with `&` | `/weather?q=Colombo&units=metric&appid=KEY` |
| **Body** | Data sent to the server (POST/PUT/PATCH); GET has no body | `{"name": "Nimal"}` |

**Parts of a response:** a **status code** (did it work?), headers, and a **body** (usually JSON).

| Status code ** | Meaning |
|---|---|
| **200 OK** | Success (GET worked) |
| **201 Created** | New resource created (POST worked) |
| **204 No Content** | Success, nothing to return (often DELETE) |
| **400 Bad Request** | Request was wrong (missing/invalid data) |
| **401 Unauthorized** | Not logged in / missing or wrong API key |
| **403 Forbidden** | Logged in but not allowed |
| **404 Not Found** | Resource does not exist (for example an invalid city) |
| **500 Internal Server Error** | The server failed |

Groups: **2xx** success, **4xx** client error, **5xx** server error.

An **API key** is a secret code that identifies your app to the API (many weather APIs require one, usually as a query parameter).

## 4. Types of web services: SOAP vs REST

**SOAP** (Simple Object Access Protocol): a strict, **XML-only messaging protocol** with formal rules. Common in banking and enterprise systems.

**REST** (Representational State Transfer): a flexible **architectural style** built on standard **HTTP**. Supports JSON, XML or plain text. Used by most modern mobile and web APIs.

| Aspect | SOAP | REST |
|---|---|---|
| Type | Protocol | Architectural style |
| Message format | **XML envelope only** | Usually lightweight **JSON** (also XML, text) |
| Statefulness | **Can maintain state** across calls | **Stateless** by design |
| Performance | XML wrapper adds overhead, slower | Lighter, faster to parse |
| Rules | Strict, formal contract | Flexible; uses URLs + HTTP methods |
| When to use | Strict, secure enterprise contracts (banking) | Most modern mobile and web APIs |

**Stateless** means the server does not remember previous requests; **each request contains all the information needed** to process it (for example the API key and parameters every time).

**REST key ideas:** resources are identified by **URLs** (`/users/5`), actions are expressed by **HTTP methods**, data is usually JSON, and requests are stateless.

## 5. HTTP request methods (map to CRUD)

| Method | CRUD | Meaning | Example |
|---|---|---|---|
| **GET** | Read | Retrieve data without changing anything | `GET /users`, `GET /users/5` |
| **POST** | Create | Send new data to create a resource | `POST /users` with JSON body |
| **PUT** | Update | **Replace** an existing resource entirely | `PUT /users/5` |
| **PATCH** | Update | Update **part** of a resource | `PATCH /users/5` with `{"email": "..."}` |
| **DELETE** | Delete | Remove a resource | `DELETE /users/5` |

### GET vs POST **

| GET | POST |
|---|---|
| **Retrieves** data | **Sends / creates** data |
| Parameters go in the **URL** (query string) | Data goes in the **request body** (JSON) |
| No request body | Has a request body |
| Safe: does not change server data | Changes server data |
| Can be cached and bookmarked; visible in URL | Not cached; data not shown in URL |
| Success usually **200 OK** | Success usually **201 Created** (or 200) |

A raw POST request looks like **:

```
POST /users HTTP/1.1
Host: api.example.com
Content-Type: application/json

{ "name": "Nimal", "email": "nimal@example.com" }
```

## 6. JSON

**JSON (JavaScript Object Notation)** is the most common format for exchanging data between apps and servers.

Why: **lightweight** (smaller than XML), **human-readable**, **language-independent**, and maps naturally to **objects, arrays and key-value pairs**.

### JSON syntax rules

- **Object** = `{ }` containing `"key": value` pairs separated by commas.
- **Array** = `[ ]` containing an ordered list of values.
- Keys are always in **double quotes**. Strings use double quotes.
- Value types: **string** `"Colombo"`, **number** `24` or `3.5`, **boolean** `true`/`false`, **null**, **object** `{ }`, **array** `[ ]`.
- No comma after the last item. No comments.

### Lecture example: a typical JSON weather response

```
{
  "city": "Colombo",
  "temperature": 24,
  "conditions": "Cloudy",
  "forecast": [
    { "day": "Mon", "high": 26 },
    { "day": "Tue", "high": 27 }
  ]
}
```

How to read it: the whole thing is one **object**. `city` and `conditions` are strings, `temperature` is a number, and `forecast` is an **array of objects**, each with `day` and `high`. So `forecast[0].day` is `"Mon"` and `forecast[1].high` is `27`.

### JSON vs XML

The same data in XML **:

```
<weather>
    <city>Colombo</city>
    <temperature>24</temperature>
    <conditions>Cloudy</conditions>
</weather>
```

| Criteria | JSON | XML |
|---|---|---|
| Syntax | Key-value pairs with `{ }` and `[ ]` | Nested opening/closing tags, like HTML |
| File size | **Smaller**, less markup | Larger, every value needs open and close tags |
| Readability | Very easy to scan | More verbose but self-descriptive |
| Parsing speed | **Faster** on mobile | Slower, more text |
| Common use | Most modern REST APIs | SOAP services, legacy systems, config files |

## 7. Parsing JSON and XML in Android

**Parsing** = reading the text response and pulling out the values you need.

| Library | Type | Notes |
|---|---|---|
| **org.json** (`JSONObject`, `JSONArray`) | JSON, **built-in** | Ships with Android; no extra dependency |
| **Gson** (Google) | JSON | Converts JSON directly into Kotlin/Java **model classes** |
| **Moshi** (Square) | JSON | Modern, Kotlin-first, null-safe |
| **XmlPullParser** | XML | Android's native, memory-efficient XML parser; **recommended** for most XML |
| **SAX** | XML | Event-driven; low memory; good for large files |
| **DOM** | XML | Loads the whole XML tree into memory; simplest but least efficient |

### Parsing the lecture weather JSON with org.json

```
val json = JSONObject(responseBody)                // responseBody = the JSON text

val city = json.getString("city")                  // "Colombo"
val temp = json.getInt("temperature")              // 24
val conditions = json.getString("conditions")      // "Cloudy"

val forecast = json.getJSONArray("forecast")       // the [ ] array
for (i in 0 until forecast.length()) {
    val dayObj = forecast.getJSONObject(i)         // each { } inside the array
    val day = dayObj.getString("day")
    val high = dayObj.getInt("high")
    Log.d("WEATHER", "$day: $high")
}
```

Methods to remember: `getString`, `getInt`, `getDouble`, `getBoolean`, `getJSONObject` (nested `{ }`), `getJSONArray` (`[ ]`), `length()`. Use ** `optString("key")` if a key might be missing (avoids an exception).

If the **whole response is an array** (for example a list of users), start with `JSONArray(responseBody)` and loop through it with `getJSONObject(i)`.

### Parsing with Gson (into data classes) **

```
data class Forecast(val day: String, val high: Int)
data class Weather(val city: String, val temperature: Int,
                   val conditions: String, val forecast: List<Forecast>)

val weather = Gson().fromJson(responseBody, Weather::class.java)
println(weather.forecast[0].day)      // Mon
```

The property names must match the JSON keys (or use `@SerializedName("json_key")`).

## 8. Consuming APIs in Android (GET and POST)

| Library | Description |
|---|---|
| **Retrofit** | **Type-safe HTTP client** that turns a REST API into a Kotlin/Java **interface**. Pairs with **Gson or Moshi** for parsing. Best fit for most modern apps. |
| **OkHttp** | Low-level HTTP client: connections, caching, interceptors. Often runs **underneath Retrofit**. |
| **Volley** | Google's earlier networking library. Simple for small request queues; less flexible for large apps today. |

**Every request follows the same pattern:** build the request → send it over HTTP → receive a status code and payload → parse the payload → update the UI.

### Two rules every networking app needs **

1. **Internet permission** in `AndroidManifest.xml` (outside `<application>`):

```
<uses-permission android:name="android.permission.INTERNET" />
```

2. **Never do network calls on the main (UI) thread.** It would freeze the app; Android throws `NetworkOnMainThreadException`. Use coroutines (`lifecycleScope.launch`, `suspend` functions) or library callbacks.

### Retrofit: complete GET and POST example **

**Step 1: dependencies** (app `build.gradle.kts`)

```
implementation("com.squareup.retrofit2:retrofit:2.11.0")
implementation("com.squareup.retrofit2:converter-gson:2.11.0")
```

**Step 2: data classes that match the JSON**

```
data class User(val id: Int? = null, val name: String, val email: String)
```

**Step 3: API interface** (each function = one endpoint)

```
interface UserApi {
    @GET("users")                                    // GET https://.../users
    suspend fun getUsers(): List<User>

    @GET("users/{id}")                               // path parameter
    suspend fun getUser(@Path("id") id: Int): User

    @GET("weather")                                  // query parameters: weather?q=Colombo&appid=KEY
    suspend fun getWeather(@Query("q") city: String,
                           @Query("appid") apiKey: String): Response<WeatherResponse>

    @POST("users")                                   // POST with a JSON body
    suspend fun createUser(@Body user: User): User

    @PUT("users/{id}")    suspend fun updateUser(@Path("id") id: Int, @Body user: User): User
    @DELETE("users/{id}") suspend fun deleteUser(@Path("id") id: Int): Response<Unit>
}
```

| Annotation | Meaning |
|---|---|
| `@GET`, `@POST`, `@PUT`, `@PATCH`, `@DELETE` | HTTP method + endpoint path |
| `@Path("id")` | Replaces `{id}` in the URL |
| `@Query("q")` | Adds `?q=value` to the URL |
| `@Body` | Object converted to JSON and sent as the request body |
| `@Header("Authorization")` | Adds a header |

**Step 4: build the Retrofit object**

```
val retrofit = Retrofit.Builder()
    .baseUrl("https://api.example.com/")            // must end with /
    .addConverterFactory(GsonConverterFactory.create())   // JSON <-> Kotlin objects
    .build()
val api = retrofit.create(UserApi::class.java)
```

**Step 5: call it in a coroutine and handle errors**

```
// GET
lifecycleScope.launch {
    try {
        val users = api.getUsers()                  // runs off the main thread
        adapter.submitList(users)                   // update UI
    } catch (e: IOException) {
        Toast.makeText(this@MainActivity, "Network error", Toast.LENGTH_SHORT).show()
    } catch (e: Exception) {
        Toast.makeText(this@MainActivity, "Error: ${e.message}", Toast.LENGTH_SHORT).show()
    }
}

// POST
lifecycleScope.launch {
    try {
        val created = api.createUser(User(name = "Nimal", email = "nimal@example.com"))
        Toast.makeText(this@MainActivity, "Created id ${created.id}", Toast.LENGTH_SHORT).show()
    } catch (e: Exception) {
        Toast.makeText(this@MainActivity, "Failed", Toast.LENGTH_SHORT).show()
    }
}
```

## Practical scenario: Weather App (GET + JSON + error handling)

Flow when the user taps **Search Weather**: read the city → **validate** it → build the request → send a **GET** request → receive the JSON → **parse** city, temperature, condition, humidity, wind speed → **display** them (values must come from the API, not hard-coded).

Code **:

```
btnSearch.setOnClickListener {
    val city = etCity.text.toString().trim()
    if (city.isEmpty()) {                                        // Case 1: empty input
        Toast.makeText(this, "Please enter a city", Toast.LENGTH_SHORT).show()
        return@setOnClickListener
    }
    lifecycleScope.launch {
        try {
            val response = api.getWeather(city, API_KEY)
            if (response.isSuccessful && response.body() != null) {
                val w = response.body()!!
                tvCity.text = w.name
                tvTemp.text = "${w.main.temp}°C"
                tvHumidity.text = "Humidity: ${w.main.humidity}%"
            } else if (response.code() == 404) {                  // Case 2: invalid city
                Toast.makeText(this@MainActivity, "City not found", Toast.LENGTH_SHORT).show()
            } else {                                              // Case 4: other API error
                Toast.makeText(this@MainActivity, "API error ${response.code()}",
Toast.LENGTH_SHORT).show()
            }
        } catch (e: IOException) {                                // Case 3: no network
            Toast.makeText(this@MainActivity, "Cannot connect. Check internet",
Toast.LENGTH_SHORT).show()
        }
    }
}
```

Error cases to handle: **empty input, invalid city (404), network error (no internet / timeout), API error (unsuccessful status code), malformed JSON / missing key.**

## 9. Testing APIs with Postman

**Postman** is a tool that lets developers **send API requests and inspect the responses without writing any app code**.

Uses:

- Verify an endpoint works **before** writing Android code.
- Inspect the exact **JSON or XML structure** the server returns (so you know which keys to parse).
- Save and share **request collections** with the team.
- Automate simple checks and repeat tests quickly.
- Create **mock servers** from a saved collection.

**How to send a GET request in Postman** **:

1. Click **New** → **HTTP Request**.
2. Choose **GET** from the method drop-down.
3. Enter the URL, for example `https://api.example.com/weather?q=Colombo&appid=KEY` (or add parameters in the **Params** tab).
4. Click **Send**.
5. Read the **status code** (for example `200 OK`), response time and the JSON **body**.

**How to send a POST request in Postman** **:

1. Choose **POST** and enter the URL (for example `https://api.example.com/users`).
2. Open **Body** → **raw** → **JSON**.
3. Type the JSON: `{ "name": "Nimal", "email": "nimal@example.com" }`
4. Check the header `Content-Type: application/json` (Postman adds it automatically for raw JSON).
5. Click **Send**; expect `201 Created` and the created object in the response.

Lecture example: `GET /api/weather/colombo → Status: 200 OK → { "city": "Colombo", "temperature": 24 }`

## 10. Mock APIs

A **mock (fake) API** returns **realistic sample data without a real backend**. Useful while the actual server is still being built.

Why use one:

- Build and test the Android **UI before the real backend is ready** (frontend and backend teams work in parallel).
- Safely **simulate error responses** such as 404 or 500 without breaking anything real.

- Keep working during development, even offline (local mock).
- Predictable, repeatable data for testing.

Limitation: a mock API **does not prove the real backend works**; you must test again after switching to the real API.

| Tool | How it works |
|---|---|
| **json-server** | Turns a simple JSON file into a full working REST API in seconds |
| **MockAPI.io** | Hosted, no-code tool to build fake endpoints in the browser |
| **Postman Mock Servers** | Generates a mock endpoint from a saved Postman collection |

json-server example **: create `db.json`

```
{ "users": [ { "id": 1, "name": "Nimal" }, { "id": 2, "name": "Kasuni" } ] }
```

Run `npx json-server db.json`. You instantly get `GET /users`, `GET /users/1`, `POST /users`, `PUT /users/1`, `DELETE /users/1` at `http://localhost:3000`. (The Android emulator reaches your computer's localhost at `http://10.0.2.2:3000/`.)

## 11. Putting it all together (lecture workflow)

1. **Design the API contract**: decide endpoints, methods and JSON shape.
2. **Test it in Postman**: send sample requests and confirm the response format.
3. **Build against a mock API**: develop the Android UI before the real backend exists.
4. **Swap in the real API**: point the app at the live endpoint (change the base URL).
5. **Parse and display**: use Retrofit/Gson (or XmlPullParser) to show the data.

## 12. Location-based services

**Location-Based Services (LBS)** use the device's geographic position to provide **context-aware features**: navigation, ride-hailing, food delivery, fitness, weather, campus services.

**How a phone knows its location: GPS/GNSS** satellite signals, plus **network-based** information (Wi-Fi and mobile cell towers). Apps do not calculate this themselves; **Android location APIs** give a common interface.

**Architecture:** Mobile app → requests location through **Android location APIs** → **Android Location Services** use available sources → returns a **Location result** (latitude, longitude, accuracy, timestamp). The app consumes a result; it does not control GPS hardware directly.

| Location data | Meaning |
|---|---|
| **Latitude** | North/south position in degrees |
| **Longitude** | East/west position in degrees |
| **Accuracy** | Estimated uncertainty in metres |
| **Timestamp** | When the location was obtained |
| Also | speed, altitude, bearing |

| Location API | Description |
|---|---|
| **LocationManager** | Android framework API for accessing location services (more manual control) |
| **Fused Location Provider (FLP)** | **Google Play services** API that **combines** GPS, Wi-Fi and cell signals to give the best location with less battery. Convenient choice for most modern apps. |

### Location permissions

Location is **protected** because it can reveal sensitive information about the user's movements.

| Permission | Gives |
|---|---|
| `ACCESS_COARSE_LOCATION` | **Approximate** location (about a city block) |
| `ACCESS_FINE_LOCATION` | **Precise** location (GPS) when available and permitted |
| `ACCESS_BACKGROUND_LOCATION` ** | Location while the app is in the background (Android 10+); stricter rules, request only if truly needed |

**Step 1: declare in AndroidManifest.xml**

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

**Step 2: request at runtime** (location is a “dangerous” permission, so since Android 6 the user must approve it while the app runs). Request **only the level of access needed**.

**Permission workflow (lecture):** App needs location → **check** if permission is already granted → if not, **request** it → user **grants or denies** → Granted: use the location API / Denied: provide an alternative or explain the limitation (do not crash).

```
private val LOCATION_REQUEST = 100

// 1. Check, and request if not granted (lecture code)
if (ActivityCompat.checkSelfPermission(this,
        Manifest.permission.ACCESS_FINE_LOCATION) != PackageManager.PERMISSION_GRANTED) {
    ActivityCompat.requestPermissions(this,
        arrayOf(Manifest.permission.ACCESS_FINE_LOCATION), LOCATION_REQUEST)
    return
}
getLocation()   // already granted

// 2. ** Handle the user's answer
override fun onRequestPermissionsResult(requestCode: Int,
        permissions: Array<out String>, grantResults: IntArray) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults)
    if (requestCode == LOCATION_REQUEST &&
        grantResults.isNotEmpty() && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
        getLocation()
    } else {
        Toast.makeText(this, "Location permission denied", Toast.LENGTH_SHORT).show()
    }
}
```

## Getting the current location (Fused Location Provider)

Workflow: 1. check permission → 2. obtain the fused client → 3. request current location (or updates) → 4. receive a Location → 5. use latitude, longitude, accuracy. Always handle **unavailable location** (null) and **permission denied**.

```
// dependency: implementation("com.google.android.gms:play-services-location:21.3.0")
val fusedClient = LocationServices.getFusedLocationProviderClient(this)

fusedClient.getCurrentLocation(Priority.PRIORITY_HIGH_ACCURACY, null)
    .addOnSuccessListener { location ->
        if (location != null) {
            tvLat.text = "Lat: ${location.latitude}"
            tvLng.text = "Lng: ${location.longitude}"
            tvAcc.text = "Accuracy: ${location.accuracy} m"
        } else {
            tvLat.text = "Location unavailable"
        }
    }
// fusedClient.lastLocation gives the last known location (faster, may be old or null)
```

**Distance between two points:** `val metres = currentLocation.distanceTo(campusLocation)`, then `if (metres <= 200) "Inside Zone" else "Outside Zone"`.

**Continuous updates** (for example a speed tracker): build a `LocationRequest` with an interval, register a `LocationCallback` with `requestLocationUpdates(...)`, and call `removeLocationUpdates(callback)` in `onPause()`/`onStop()` to save battery. `location.speed` gives m/s.

### Google Maps vs Location API

| Location API | Google Maps SDK |
|---|---|
| Provides latitude, longitude, accuracy | **Displays** a map |
| Used for decisions, tracking, distance, logic | Markers, camera position, zoom, visual interaction |

An app can use either alone or combine both (location gives the position, Maps shows it). Google Maps needs an **API key**.

### Geofencing

A **geofence** is a **virtual boundary** (centre coordinates + radius) around a real place. Instead of asking for location continuously, the app **reacts to events** when the user crosses the boundary.

- Transitions: **ENTER**, **EXIT**, **DWELL** (stays inside for some time).
- Steps: define the region → register the geofence (`GeofencingClient`, `GeofencingRequest`, `PendingIntent`) → location services monitor the boundary → transition happens → app receives the event (BroadcastReceiver) and acts (notification, attendance).
- Examples: campus attendance, store notifications, delivery zones, smart-home automation.
- Geofencing is **event-driven**, unlike continuously displaying location.

### Design considerations for location and sensors

| Concern | Guidance |
|---|---|
| **Privacy** | Request only necessary access and explain why |
| **Battery** | Avoid unnecessary continuous updates and high-frequency sampling |
| **Availability** | Not every device has every sensor |
| **Accuracy** | Readings are measurements, not perfect values |
| **Lifecycle** | Register/unregister listeners and stop updates appropriately |
| **Resilience** | Handle denied permissions, missing sensors and missing location |

## 13. Gestures and smartphone sensors

In the slides, gestures appear as **“gesture-based interaction”** using the **accelerometer** (shake). Exams may also ask about **touch gestures**, so both are covered here.

### Touch gestures **

A **gesture** is a touch interaction the user performs on the screen.

| Gesture | Description | Typical use |
|---|---|---|
| **Tap** (single tap) | Quick touch and release | Select, press a button |
| **Double tap** | Two quick taps | Zoom in, “like” a photo |
| **Long press** | Touch and hold | Context menu, select item, delete option |
| **Swipe / Fling** | Fast slide in a direction | Delete a list item, change page, dismiss |
| **Scroll / Drag** | Slow slide while touching | Scroll a list, move an object |
| **Pinch** (two fingers) | Fingers apart / together | Zoom in / out (maps, photos) |

**Simple listeners:**

```
button.setOnClickListener { /* tap */ }
button.setOnLongClickListener {
    Toast.makeText(this, "Long pressed", Toast.LENGTH_SHORT).show()
    true                          // true = event handled
}
```

**Raw touch events with OnTouchListener (MotionEvent):**

```
view.setOnTouchListener { v, event ->
    when (event.action) {
        MotionEvent.ACTION_DOWN -> Log.d("TOUCH", "Finger down at ${event.x}, ${event.y}")
        MotionEvent.ACTION_MOVE -> Log.d("TOUCH", "Finger moving")
        MotionEvent.ACTION_UP   -> Log.d("TOUCH", "Finger lifted")
    }
    true
}
```

**GestureDetector** recognises common gestures for you:

```
class MainActivity : AppCompatActivity() {
    private lateinit var gestureDetector: GestureDetector

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        gestureDetector = GestureDetector(this, object : GestureDetector.SimpleOnGestureListener() {
            override fun onDown(e: MotionEvent) = true      // must return true to get other events
            override fun onSingleTapUp(e: MotionEvent): Boolean { show("Single tap"); return true }
            override fun onDoubleTap(e: MotionEvent): Boolean { show("Double tap"); return true }
            override fun onLongPress(e: MotionEvent) { show("Long press") }
            override fun onFling(e1: MotionEvent?, e2: MotionEvent,
                                 velocityX: Float, velocityY: Float): Boolean {
                if (e1 != null && e2.x > e1.x) show("Swipe right") else show("Swipe left")
                return true
            }
        })
    }

    // pass every touch event to the detector
    override fun onTouchEvent(event: MotionEvent): Boolean {
        return gestureDetector.onTouchEvent(event) || super.onTouchEvent(event)
    }

    private fun show(msg: String) = Toast.makeText(this, msg, Toast.LENGTH_SHORT).show()
}
```

Pinch-to-zoom uses `ScaleGestureDetector` (`detector.scaleFactor` > 1 = zoom in). Swipe-to-delete in a RecyclerView uses `ItemTouchHelper`.

**Gesture design guidance:** use gestures users already know; keep a **visible alternative** (a button) for essential actions because gestures are hidden; give feedback (ripple, animation); keep targets at least 48dp.

### Smartphone sensors

Sensors measure **movement, position or environmental conditions**: accelerometer, gyroscope, proximity, light, magnetic field. Android provides one common **Sensor Framework**. Always **check the sensor exists** first.

| Sensor API class | Role |
|---|---|
| **SensorManager** | Manages access to available sensors |
| **Sensor** | Represents one specific sensor |
| **SensorEvent** | Contains the readings (`values[]`) and a timestamp |
| **SensorEventListener** | Receives value changes (`onSensorChanged`) and accuracy changes (`onAccuracyChanged`) |

| Sensor | Measures | Use |
|---|---|---|
| **Accelerometer** | **Linear acceleration** along X (left/right), Y (up/down), Z (toward/away from screen); includes gravity | Shake detection, tilt, games, step/activity features |
| **Gyroscope** | **Angular velocity** (how fast it rotates) around X, Y, Z | Games, AR, orientation |
| **Proximity** | Whether an object is **near** the screen (distance or near/far) | Turn screen off during a phone call |

**Sensor code pattern (7 steps):** get SensorManager → get the Sensor → check it exists → register listener → read SensorEvent values → use them → **unregister** when not needed.

Code **:

```
class MainActivity : AppCompatActivity(), SensorEventListener {
    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
        if (accelerometer == null) Toast.makeText(this, "No accelerometer", Toast.LENGTH_SHORT).show()
    }
    override fun onResume() {
        super.onResume()
        accelerometer?.also { sensorManager.registerListener(this, it,
SensorManager.SENSOR_DELAY_NORMAL) }
    }
    override fun onPause() {
        super.onPause()
        sensorManager.unregisterListener(this)          // save battery
    }
    override fun onSensorChanged(event: SensorEvent) {
        val x = event.values[0]; val y = event.values[1]; val z = event.values[2]
        val force = sqrt(x * x + y * y + z * z)
        if (force > 15) Toast.makeText(this, "Shake detected", Toast.LENGTH_SHORT).show()
    }
    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
}
```

**Camera** is accessed through a different API (**CameraX**), needs the CAMERA permission and lifecycle handling (QR scanning, document capture, AR). Camera = visual data; Sensor Framework = measurements.

**Integrated campus app (lecture):** Google Maps shows the campus; Fused Location Provider gets the student's position; Geofencing detects entering/leaving campus; Accelerometer detects a shake interaction. Together the app responds to both **place** and **device context**.

## 14. Practice questions with short model answers

**API vs web service?** An API is any interface for requesting functionality, with or without a network; a web service is an API that always works over a network using a protocol like SOAP or REST.

**Why is REST called stateless?** The server keeps no client session between requests; each request contains all the information needed to process it.

**SOAP vs REST (one difference)?** SOAP uses only XML envelopes with strict rules; REST usually uses lightweight JSON over standard HTTP methods.

**GET vs POST?** GET retrieves data with parameters in the URL and no body; POST sends data in the request body to create a resource.

**Which HTTP method updates only one field of a record?** PATCH.

**Why is JSON preferred over XML on mobile?** It is smaller and faster to parse, which saves data and battery.

**From the forecast JSON, write Kotlin to get the first day.**
`json.getJSONArray("forecast").getJSONObject(0).getString("day")`

**Why use Postman?** To send requests and inspect status codes and JSON responses before writing app code.

**What is a mock API and why use it?** A fake endpoint returning sample data so the app UI can be built and tested before the real backend is ready, including error cases.

**What permission must every app that calls an API declare?** `android.permission.INTERNET` in the manifest.

**Why must network calls not run on the main thread?** They are slow and would freeze the UI; Android throws NetworkOnMainThreadException.

**COARSE vs FINE location?** COARSE gives approximate location; FINE gives precise GPS location.

**Why request location permission at runtime and handle denial?** Location is sensitive personal data, so the user must approve it; if denied the app must still work and explain the limitation.

**Location API vs Google Maps?** The Location API provides coordinates; Google Maps displays them visually on a map.

**What is geofencing?** A virtual boundary that triggers ENTER, EXIT or DWELL events when the user crosses it.

**Accelerometer vs gyroscope?** Accelerometer measures linear movement/acceleration; gyroscope measures rotation speed.

**Name the class used to detect swipe and double-tap gestures.** GestureDetector (with SimpleOnGestureListener).

**Why unregister sensor listeners in onPause()?** To save battery and avoid processing readings when the screen is not visible.